SCRUM-160 — Guía de entorno: Node, Expo y expo-router

Este documento recoge lo decidido esta semana por el gestor de calidad sobre la configuración técnica del repositorio (tarea SCRUM-160: ESLint, Prettier y CI/CD). Antes de llegar a eso, hubo que resolver el entorno base: no había `package.json`, y al generar el proyecto con Expo apareció una incompatibilidad de versión de Node que afecta a todo el equipo, no solo a quien lo instaló primero.

Objetivo: que cualquier persona del equipo pueda clonar el repositorio, seguir estos pasos y tener el proyecto corriendo sin tropezar con los mismos problemas.

1. Instalación de Node 22 (obligatorio)

1. Instalación de Node 22 (obligatorio)

El framework elegido (Expo, ver sección 2) necesita Node 22.13 o superior a partir de su SDK 57. Las versiones de Node más antiguas (por ejemplo, 18 o 20) dan errores de instalación o avisos `EBADENGINE` que pueden provocar fallos difíciles de depurar más adelante. **Todo el equipo debe actualizar a Node 22 antes de tocar el proyecto.**

La forma recomendada es usar `nvm` (Node Version Manager), que permite tener varias versiones de Node instaladas sin romper nada del sistema ni afectar a otros proyectos.

1. Comprobar si ya tienes `nvm` instalado:

   ```
   nvm --version
   ```

   Si da error de "comando no encontrado", instálalo:

   ```
   curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
   source ~/.bashrc
   ```

2. Instalar y activar Node 22:

   ```
   nvm install 22
   nvm use 22
   ```

3. Comprobar que ha quedado activo:

   ```
   node -v
   ```

Debe mostrar algo como `v22.x.x` (en el repo se está usando `v22.23.4`).

4. Dentro de la carpeta del proyecto, generar el archivo `.nvmrc` (ya está subido al repo, no hace falta repetirlo, pero así es como se generó):

   ```
   node -v > .nvmrc
   ```

   Este archivo fija la versión de Node del proyecto. Si en el futuro usas `nvm` y entras a la carpeta, puedes ejecutar `nvm use` (sin número) y cogerá automáticamente la versión correcta del `.nvmrc`.

Si tienes varios gestores de versiones de Node instalados (nvm, fnm, volta...): pueden pisarse entre sí. Si después de `nvm use 22` (o equivalente) `node -v` sigue mostrando una versión antigua, comprueba cuál gana prioridad en tu `PATH`:

```
which -a node
echo $PATH
```

El primero de la lista es el que se usa de verdad. Usa el gestor que aparezca primero en tu `PATH` para fijar la versión 22 como predeterminada (por ejemplo, con `fnm`: `fnm install 22 && fnm default 22`; con `nvm`: `nvm alias default 22`), y verifica en una terminal **nueva** que `node -v` ya devuelve 22 sin tener que escribir nada más.

Importante: si ya tenías `node_modules` o `package-lock.json` instalados con una versión antigua de Node, bórralos y reinstala limpio después de cambiar de versión:

```
rm -rf node_modules package-lock.json
npm install
```

2. ¿Qué es React Native y Expo?

2. ¿Qué es React Native y Expo?

El equipo tenía decidido inicialmente Flutter (Dart). Esa decisión se ha descartado: la app se construirá con **React Native**, gestionado con **Expo**.

React Native es un framework que permite escribir la app en JavaScript/TypeScript y que compile a app nativa de Android (y iOS, aunque no es el objetivo aquí). La interfaz se construye con componentes declarativos similares a los de una página web moderna.

Expo es una capa sobre React Native que simplifica mucho el día a día:

- No hace falta configurar Android Studio ni Gradle a mano para empezar a desarrollar.

- Se puede probar la app en una tablet real instalando la app **Expo Go** (desde la Play Store) y escaneando un código QR — sin cables ni compilar un APK.
- Incluye comandos (`expo install`) que instalan automáticamente la versión de cada paquete compatible con la versión de Expo del proyecto, evitando desajustes de versiones entre dependencias nativas.

¿Por qué React Native + Expo y no Flutter o nativo (Kotlin)?

- El backend del proyecto ya está decidido: **Supabase + Vercel**, que son herramientas del ecosistema JavaScript/TypeScript con mejor soporte y documentación para React Native que para Dart.
- Expo simplifica el despliegue y las pruebas en tablets Android, que es el dispositivo objetivo del colegio.
- Hay más literatura y librerías de accesibilidad ya hechas para React Native (relevante porque el proyecto es una app accesible para alumnado con discapacidad intelectual).
- Ir "a pelo" en Kotlin nativo sería más lento de desarrollar con un equipo de 6 personas rotando roles y con plazos de sprint ajustados, y no aporta ninguna ventaja real para este tipo de app (una agenda, no un juego ni algo que necesite máximo rendimiento gráfico).

3. ¿Qué es expo-router?

3. ¿Qué es expo-router?

Es el sistema de navegación oficial de Expo, basado en **file-based routing** (como Next.js, pero para apps móviles): cada archivo dentro de una carpeta `app/` es automáticamente una pantalla, sin tener que configurar a mano un "stack navigator".

Ejemplo de estructura:

```
app/
  index.tsx       → pantalla de inicio ("/")
  agenda.tsx       → pantalla "/agenda"
  perfil.tsx       → pantalla "/perfil"
  _layout.tsx      → layout compartido (cabecera, menú, etc.)
```

Por qué lo usamos en este proyecto:

- Con 6 personas tocando el repo, cada una puede trabajar en su propia pantalla (archivo) sin pisarse tanto en la configuración de navegación.
- Es el estándar actual recomendado por Expo — mejor soporte y documentación a futuro.

- Encaja bien con una app dividida en secciones claras (agenda, perfil del alumno, vista del tutor...).

Alternativa descartada: `@react-navigation/native` instalado y configurado a mano dentro de un único `App.tsx`. Es más control manual, pero mucho más código de configuración — no compensa para este proyecto.

Importante: al usar expo-router, la estructura del proyecto cambia: en vez de un único `App.tsx` como punto de entrada, las pantallas van dentro de la carpeta `app/`. Si alguien ya tiene el proyecto generado sin expo-router, hay que reestructurarlo (ver sección 4).

4. Cómo replicar el entorno (pasos para cada miembro del equipo)

4. Cómo replicar el entorno (pasos para cada miembro del equipo)

1. Clonar el repositorio (si no lo tienes ya) y situarte en la carpeta del proyecto:

```
git clone <url-del-repo>
cd agenda-personalizada
git checkout main
git pull
```

2. Instalar/activar Node 22 (ver sección 1 si no lo tienes hecho):

```
nvm install 22
nvm use 22
node -v   # debe mostrar v22.x.x
```

3. Instalar las dependencias del proyecto:

```
npm install
```

   Si aparecen avisos `EBADENGINE`, es que no estás en Node 22 — revisa el paso 2.

4. Arrancar el proyecto:

```
npm run android   # abre en emulador Android o dispositivo conectado
npm run web       # abre en navegador, útil para pruebas rápidas de UI
```

   Para probar en una tablet física: instala la app **Expo Go** desde la Play Store en la tablet, ejecuta `npx expo start` en el ordenador y escanea el código QR que aparece en la terminal.

Nota: no borres ni edites `package-lock.json` a mano. Si necesitas añadir una dependencia nueva, edita `package.json` (o usa `npm install <paquete>` / `npx expo install <paquete>` para paquetes nativos) y sube ambos archivos juntos en el mismo commit.

5. Archivos del repositorio: qué son y cuáles se pueden borrar

5. Archivos del repositorio: qué son y cuáles se pueden borrar

Archivo	Qué es	¿Se puede borrar?
`package.json`	Declara las dependencias del proyecto (versiones flexibles, ej. `^51.0.0`) y los scripts (`npm run ...`)	No
`package-lock.json`	Fija la versión exacta instalada de cada paquete y sus subdependencias, para que todo el equipo instale exactamente lo mismo	No — nunca se borra manualmente ni se edita a mano; se actualiza solo con `npm install`
`.nvmrc`	Fija la versión de Node del proyecto (`v22.23.4`)	No
`App.tsx` / `index.ts`	Punto de entrada actual de la app (placeholder generado por Expo). Se sustituirán por la carpeta `app/` al migrar a expo-router	Se modifican/reestructuran, no se borran sin más
`LICENSE`	Licencia de software genérica (normalmente MIT) que Expo añade por defecto	Sí, si no aplica a un proyecto académico
`AGENTS.md`	Instrucciones de contexto para asistentes de IA tipo Codex	Sí, si no se usa ese asistente
`CLAUDE.md`	Instrucciones de contexto específicas para Claude Code	Sí, si no se usa esa herramienta

Si alguien del equipo borra `LICENSE`, `AGENTS.md` o `CLAUDE.md`, no afecta al funcionamiento de la app — son solo metadatos del repositorio.

6. Pendiente en SCRUM-160

6. ESLint, Prettier, VS Code y CI/CD (ya configurados)

ESLint y Prettier están configurados y pasando limpio en el repo. Si clonas o actualizas el proyecto, después de `npm install` (sección 4) puedes comprobar que todo está en orden con:

```
npm run lint
npm run format:check
```

Si `npm run lint` no muestra nada tras el comando, está todo correcto (sin salida = sin errores/avisos).

Nota sobre versiones: usamos ESLint **9**, no la 10. ESLint 10 es demasiado reciente y algunos plugins (como `eslint-plugin-react`) todavía no son compatibles — si alguna vez `npm install` sube ESLint a la 10 por accidente, bajarlo así:

```
npm install eslint@^9 --save-dev --legacy-peer-deps
```

VS Code: al abrir el proyecto en VS Code por primera vez, te sugerirá instalar dos extensiones (recomendadas en `.vscode/extensions.json`): **ESLint** y **Prettier**. Instálalas — con eso, el formateo automático al guardar (`Ctrl+S`) y el aviso de errores de lint en el editor ya están configurados para todo el equipo sin que tengas que tocar nada (`.vscode/settings.json` ya lo trae hecho).

CI/CD (GitHub Actions): cada vez que se abra un Pull Request contra `main` (o se haga push a `main`), GitHub ejecuta automáticamente `npm run lint` y `npm run format:check`. Si alguno falla, el PR se marca en rojo y **no debería mergearse** hasta corregirlo. Esto es justo el control de calidad que evita que código mal formateado o con errores de estilo llegue a `main`.

Si tu PR sale en rojo:

```
npm run lint:fix     # arregla automáticamente lo que pueda
npm run format       # formatea todo con Prettier
```

Y vuelve a hacer commit/push de los cambios.

6. Pendiente en SCRUM-160

- [x] ~~Reestructurar el proyecto a la carpeta `app/` con expo-router~~
- [x] ~~Instalar y configurar ESLint (con `eslint-plugin-jsx-a11y`)~~

- [x] ~~Añadir `.prettierrc` y `.prettierignore`~~
- [x] ~~Scripts de lint/format en `package.json`~~
- [x] ~~Ajustes de VS Code (`.vscode/settings.json` y `.vscode/extensions.json`)~~
- [x] ~~CI/CD con GitHub Actions (lint + format en cada PR)~~
- [ ] Conectar `@supabase/supabase-js` con las credenciales del proyecto
- [ ] Decidir si se mantiene o se borra `LICENSE` / `AGENTS.md` / `CLAUDE.md`
- [ ] Hacer commit y push de toda esta configuración a una rama (no directamente a `main`, salvo que el equipo decida lo contrario)

Cualquier duda sobre estos pasos, comentar en este documento o preguntar directamente al gestor de calidad.